
Git Ops Cheatsheet

The 20 commands you actually use, plus the "oh no" recovery section.

GitHub ↗

You know Git. This isn't a tutorial — it's the reference for the commands you reach for under pressure, and the recovery moves that turn "I just destroyed three hours of work" into "nothing happened." The headline truth: **almost nothing in Git is actually gone.** `reflog` is your time machine. Learn it and you stop fearing the tool.

SAFE routine, no data-loss risk **REWRITES HISTORY** changes commits — know your branch

RECOVERY emergency / destructive-risk territory

DAILY DRIVER

SAFE

The loop you run a hundred times a day.

COMMAND	ACTION
<code>git status -sb</code>	short, with branch info
<code>git add -p</code>	stage hunks interactively — review what you commit
<code>git commit -m "msg"</code>	commit staged changes
<code>git commit --amend</code>	fix the last commit (message or staged content)
<code>git commit --amend --no-edit</code>	tack staged changes onto last commit, keep message
<code>git pull --rebase</code>	avoid the noise merge commits on pull
<code>git push</code>	push to the tracked remote branch
<code>git push -u origin feature/x</code>	set upstream on first push
<code>git log --oneline --graph --all --decorate</code>	the only log you need

BRANCHING

SAFE

`git switch` replaces the overloaded `checkout` for branch operations — clearer intent, fewer footguns.

COMMAND	ACTION
<code>git switch -c feature/x</code>	create + switch (modern; replaces checkout -b)
<code>git switch main</code>	switch (replaces checkout <branch>)
<code>git branch -d feature/x</code>	delete merged branch
<code>git branch -D feature/x</code>	force-delete unmerged
<code>git branch -vv</code>	show tracking + last commit per branch
<code>git fetch -p</code>	fetch + prune dead remote branches

STASH

SAFE

Shelve dirty work without committing it.

COMMAND	ACTION
<code>git stash push -m "wip auth"</code>	named stash
<code>git stash list</code>	list all stashes
<code>git stash show -p stash@{0}</code>	diff a stash
<code>git stash pop</code>	apply + drop
<code>git stash apply stash@{1}</code>	apply, keep in list
<code>git stash drop stash@{0}</code>	delete a stash entry
<code>git stash -u</code>	include untracked files

REBASE & HISTORY SURGERY

REWRITES HISTORY

Fine — but only on branches nobody else has pulled from. Once it's pushed and shared, reach for `revert` instead.

COMMAND	ACTION
<code>git rebase main</code>	replay current branch onto main
<code>git rebase -i HEAD~5</code>	squash/reword/reorder last 5 commits
<code>git rebase --abort</code>	bail out of a bad rebase
<code>git rebase --continue</code>	after resolving conflicts
<code>git cherry-pick <sha></code>	grab one commit onto current branch
<code>git revert <sha></code>	safe undo: new commit that reverses <sha>

THE "OH NO" SECTION RECOVERY

`git reflog` logs every HEAD movement for ~90 days by default. Whatever you just did, it's almost certainly still reachable.

See every HEAD movement — your undo history

```
git reflog                # EVERY HEAD movement – your undo history
git reset --hard HEAD@{2}  # jump back to where HEAD was 2 moves ago
```

Undo the last commit — pick your blast radius

```
git reset --soft HEAD~1    # keep changes staged
git reset --mixed HEAD~1   # keep changes unstaged (working dir)
git reset --hard HEAD~1    # nuke commit AND changes (careful)
```

Recover a “deleted” branch

```
git reflog | grep feature/x    # find the sha
git switch -c feature/x <sha>
```

Recover a single file to a previous state

```
git restore --source=HEAD~3 path/to/file
git checkout <sha> -- path/to/file    # older syntax, still works
```

Accidentally committed to main instead of a branch

```
git switch -c feature/x        # branch now points at your commits
git switch main
git reset --hard origin/main    # reset main back to clean
```

Threw away uncommitted work with reset --hard? If it was ever staged:

```
git fsck --lost-found          # dangling blobs live here
```

UNDO A PUSHED COMMIT RECOVERY

Two very different moves depending on whether anyone else has that history.

Safe (public / shared branch): revert creates a new undo commit

```
git revert <sha> && git push
```

Rewriting pushed history (only on branches you own, never shared main)

```
git reset --hard <good-sha>  
git push --force-with-lease    # NOT --force – lease checks nobody else pushed
```

CONFIG & ALIASES

SAFE

Run once per machine.

CONFIG	WHY
<code>git config --global pull.rebase true</code>	always rebase on pull, no merge-commit noise
<code>git config --global init.defaultBranch main</code>	new repos default to main
<code>git config --global alias.lg "log --oneline --graph --all --decorate"</code>	git lg shortcut
<code>git config --global alias.st "status -sb"</code>	git st shortcut
<code>git config --global rerere.enabled true</code>	remember conflict resolutions

THINGS THAT BITE YOU

READ THIS ONCE



`--force` vs `--force-with-lease` — always use the lease version. Plain `--force` will happily overwrite a teammate's pushed work without warning.



`reset --hard` discards uncommitted changes permanently — there's no reflog for unstaged working-dir edits. Stash first if unsure.



Detached HEAD isn't broken — you're just not on a branch. Commits made there get garbage-collected unless you `switch -c` to keep them. `reflog` finds them if you wandered off.



`git pull` without `--rebase` — spawns merge commits that clutter history. Set `pull.rebase true` globally.



Line-ending wars (CRLF/LF) — across Windows/Mac/Linux — set `core.autocrlf` consistently per platform, or use a `.gitattributes` with `* text=auto`.